



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

Lezione 6 - Algoritmi di ricerca locale (1)



Nelle lezioni precedenti...

Agenti Risolutori di Problemi

Esaminare le azioni future: che significa?

Ipotizziamo che l'ambiente sia **osservabile**, pertanto l'agente conosce *sempre* lo stato corrente del mondo. Nell'esempio precedente, questo significherebbe che ogni città abbia un cartello che indichi la sua presenza ai guidatori in arrivo.

Ipotizziamo che l'ambiente sia **discreto**, pertanto in qualsiasi stato esiste un numero finito di azioni tra cui scegliere.

Ipotizziamo che l'ambiente sia **noto**, pertanto l'agente saprà sempre quali stati saranno raggiunti da ciascuna azione. Nei problemi di navigazione, questo requisito è soddisfatto a patto che si abbia a disposizione una mappa accurata.

Ipotizziamo che l'ambiente sia **deterministico**, pertanto ogni azione avrà uno ed un solo risultato.

Sotto queste ipotesi, la soluzione di qualsiasi problema è rappresentata da una **sequenza fissata di azioni**. In altri termini, una soluzione potrebbe essere implementata come una strategia ramificata che raccomandi azioni future diverse in base alle percezioni giunte.

In una situazione del genere, l'agente in esempio saprebbe esattamente dove si troverà dopo la prima azione e che cosa percepirà, e così via. Così facendo, potrebbe identificare una soluzione specificata da una sequenza di azioni che porti da una città all'altra fino a Bucarest.

Verso la ricerca locale

In questa parte del corso, inizieremo a rilassare queste ipotesi in maniera da avvicinarci ad una formulazione più reale dei problemi di ricerca.

Gli algoritmi che abbiamo studiato fino adesso sono progettati per esplorare sistematicamente lo spazio degli stati: per farlo, tengono in memoria uno o più cammini e registrano quali alternative sono state esplorate in ogni punto del cammino.

Quando viene raggiunto uno stato obiettivo, **il cammino** verso quello stato costituisce una soluzione del problema.

Tuttavia, in molti casi il cammino che porta alla soluzione è *irrilevante*; ad esempio, nel problema delle N regine ciò che conta è la disposizione finale delle regine sulla scacchiera, non l'ordine con cui sono state aggiunte.

Questo vale per molti problemi reali: la configurazione degli spazi nelle fabbriche, la programmazione automatica degli orari di lavoro, la gestione di portafogli azionari, l'ottimizzazione di reti di telecomunicazione, e altri.

Nel contesto di questa classe di problemi, possiamo considerare algoritmi diversi che ignorano il cammino che porta alla soluzione.

Algoritmi di Ricerca Locale

Algoritmi di ricerca locale

Quindi, in molti problemi di ottimizzazione lo **stato obiettivo è esso stesso la soluzione** al problema, indipendentemente da come ci si è arrivati.

Lo spazio degli stati è dato dall'insieme delle configurazioni “complete”, ovvero quelle che portano alla risoluzione del problema e, quindi, ad uno stato obiettivo.

In tali casi, si possono utilizzare **algoritmi di miglioramento iterativo**, i quali mantengono in memoria solo lo stato corrente e tentano di migliorarlo.

Algoritmi di ricerca “tradizionali”

Considerano interi cammini dallo stato iniziale a quello obiettivo

Hanno una complessità temporale e spaziale generalmente esponenziale

Non possono essere applicati in problemi con spazio degli stati grandi/infiniti

Algoritmi di ricerca locale

Considerano lo stato corrente con l'obiettivo di migliorarlo

Usano poca memoria, molto spesso avendo una complessità costante

Possono trovare soluzioni ragionevoli (sub-ottimali) a problemi complessi

Algoritmi di Ricerca Locale

Algoritmi di ricerca locale - La funzione obiettivo

Gli algoritmi di ricerca locale **non hanno un test obiettivo**, ma affidano le loro azioni ad una **funzione obiettivo** che indica quanto la ricerca sta migliorando nelle iterazioni.

Questo li rende adatti alla risoluzione di classici problemi di ottimizzazione, come ad esempio la disposizione dei tavoli in un ristorante. In questi casi, infatti, non esiste un vero e proprio test obiettivo.

La scelta della funzione obiettivo è fondamentale per l'efficienza del sistema e definisce *l'insieme delle soluzioni che possono essere raggiunte da una soluzione s in un singolo passo di ricerca dell'algoritmo*.

Tipicamente, la funzione obiettivo è definita attraverso le possibili mosse che l'algoritmo può effettuare. È da notare che la ricerca locale si basa sull'esplorazione iterativa delle “soluzioni vicine” che possono migliorare quella corrente mediante modifiche locali.

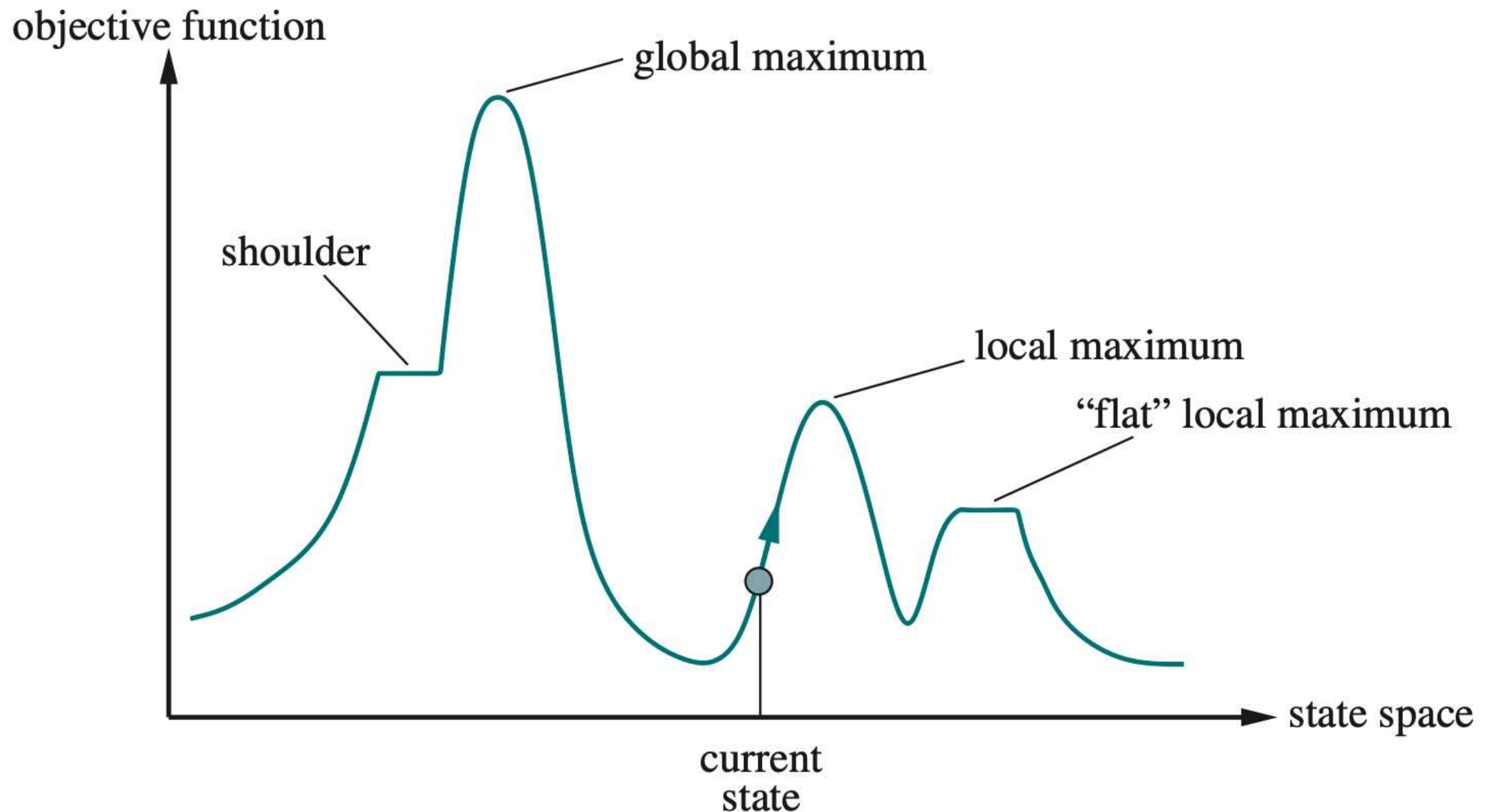
Struttura dei vicini (neighborhood): Una struttura dei vicini è una funzione F che assegna a ogni soluzione s dell'insieme di soluzioni S un insieme di soluzioni $N(s)$ sottoinsieme di S .

È importante notare che la soluzione trovata da un algoritmo di ricerca locale non è detto che sia **globalmente ottima**, ma può essere **ottima rispetto ai cambiamenti locali**. In altri termini, la soluzione può essere la migliore possibile date le circostanze.

Algoritmi di Ricerca Locale

Algoritmi di ricerca locale - Il panorama dello spazio degli stati

Per poter meglio comprendere il funzionamento degli algoritmi di ricerca locale, è utile visualizzare lo spazio degli stati in maniera diversa rispetto ad un albero.



Algoritmi di Ricerca Locale

Spiegazione grafico:

Questo grafico rappresenta **lo spazio degli stati visto come un paesaggio collinare**, dove:

Asse orizzontale (state space): sono tutti gli stati possibili che l'algoritmo può visitare.

Spostarsi a destra o sinistra significa passare da uno stato a un altro vicino.

Asse verticale (objective function): misura quanto è “buono” quello stato, cioè il valore della funzione obiettivo che vogliamo massimizzare (o minimizzare).

Ora i termini:

Current state: è il punto in cui si trova attualmente l'algoritmo. Da lì decide se salire o scendere nel paesaggio cercando uno stato migliore.

Global maximum: lo stato migliore in assoluto, la cima più alta di tutto il paesaggio. È la soluzione ottimale.

Local maximum: una cima più piccola, che sembra buona se guardi solo intorno, ma non è la migliore di tutto lo spazio. Qui gli algoritmi locali rischiano di bloccarsi.

“Flat” local maximum: un altopiano, cioè una zona piatta con più stati vicini aventi lo stesso valore. Qui la ricerca può fermarsi senza sapere dove andare.

Shoulder (spalla): un punto in cui il paesaggio smette di salire ma non è ancora una cima vera. Può trarre in inganno perché sembra un massimo, ma in realtà basta continuare un po' per trovare un valore più alto.

In poche parole: gli algoritmi di **ricerca locale** (come Hill Climbing o Simulated Annealing) sono come scalatori ciechi che cercano di salire sempre più in alto, ma rischiano di fermarsi su un'altura sbagliata (local maximum o shoulder) invece che sulla montagna più alta (global maximum).

Algoritmi di Ricerca Locale

Algoritmi di ricerca locale - Il panorama dello spazio degli stati

Per poter meglio comprendere il funzionamento degli algoritmi di ricerca locale, è utile visualizzare lo spazio degli stati in maniera diversa rispetto ad un albero.

Un algoritmo di ricerca locale **completo** trova sempre un obiettivo, se questo esiste.

Un algoritmo di ricerca locale **ottimo** trova sempre un minimo/massimo globale.

Più formalmente, un massimo (minimo) *locale* è una soluzione s tale che, data una funzione di valutazione f , avveri la seguente equazione:

$$\forall s' \in N(s), f(s) \geq f(s')$$

Quando risolviamo un problema di massimizzazione (minimizzazione) cerchiamo un massimo (minimo) *globale*, s_{opt} , cioè una soluzione tale che

$$\forall s, f(s_{opt}) \geq f(s)$$

Banalmente, più è largo il neighborhood più è probabile che un massimo/minimo locale sia anche globale -> più è largo il neighborhood e migliore sarà qualità della soluzione.

Algoritmi di Ricerca Locale

L'algoritmo Hill-Climbing

L'algoritmo Hill-Climbing è il più semplice tra gli algoritmi di ricerca locale e ha l'obiettivo di “scalare” lo spazio di ricerca per trovare un massimo/minimo globale.

```
function HILL-CLIMBING(problema) returns uno stato che è un massimo  
  
  nodo_corrente ← CREA-NODO(problema.STATO-INIZIALE)  
  
  loop do  
  
    vicino ← il successore di nodo_corrente di valore più alto  
    if vicino.VALORE ≤ nodo_corrente.VALORE then return nodo_corrente.STATO  
    nodo_corrente ← vicino
```

Algoritmi di Ricerca Locale

Spiegazione pseudocodice:

Questo pseudocodice descrive l'algoritmo **Hill-Climbing**, il più semplice tra gli algoritmi di ricerca locale. L'idea è di "scalare" sempre verso stati migliori finché non trovi più nulla di superiore.

Ecco il funzionamento semplificato:

Nodo iniziale: si parte da uno stato qualsiasi (punto del paesaggio).

Vicino migliore: si guarda tra gli stati vicini (successori) e si sceglie quello col valore più alto (cioè migliore secondo la funzione obiettivo).

Decisione:

Se nessun vicino è migliore dell'attuale → fermati: sei su un massimo locale.

Altrimenti → spostati nel vicino migliore e ripeti.

In pratica è come scalare una montagna al buio, sempre andando nella direzione che sale di più sotto i piedi.

Limite: Hill-Climbing rischia di fermarsi:

su un **massimo locale** (una collina, non la cima più alta),

su un **plateau** (zona piatta),

su una **spalla** (pendenza che sembra un massimo).

È veloce e semplice, ma non garantisce di trovare il massimo globale.

NOTE: Negli algoritmi di **Hill-Climbing** possono verificarsi diversi problemi legati alla forma dello spazio di ricerca.

Se l'algoritmo raggiunge la vicinanza di un **massimo locale**, sarà naturalmente attratto verso quel picco, ma finirà per restare bloccato lì, senza la possibilità di esplorare altre zone potenzialmente migliori.

Quando invece incontra un **plateau** (una regione piatta), rischia di fermarsi perché non trova miglioramenti nella soluzione corrente, restituendo così un risultato sub-ottimale.

Infine, le cosiddette **creste (ridge)** sono porzioni dello spazio con brusche variazioni, dove l'algoritmo si trova a scalare una serie di massimi locali consecutivi: in queste situazioni uscire dalla trappola diventa particolarmente difficile.

Algoritmi di Ricerca Locale

L'algoritmo Hill-Climbing, un esempio

Consideriamo il problema delle 8 regine, nella sua formulazione a stato completo - ogni stato ha 8 regine sulla scacchiera, una per colonna.

La funzione successore restituisce tutti gli stati possibili generati muovendo una singola regina in un'altra casella della stessa colonna - in questo modo, ogni stato avrà $8 \times 7 = 56$ possibili successori.

La funzione obiettivo è data dal numero di coppie di regine che si stanno attaccando a vicenda, in maniera diretta o indiretta. Vogliamo chiaramente minimizzare la funzione.

Quindi, il minimo globale è 0, che si verifica solo nel caso delle soluzioni.

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

La figura rappresenta uno stato del problema con $h=17$. Sono indicati i valori della funzione di ogni possibile successore ottenuto muovendo una regina nella sua colonna - le mosse migliori sono evidenziate.

Quando ci sono più successori con lo stesso valore di funzione obiettivo, allora l'algoritmo sceglierà casualmente la prossima mossa.

Tipicamente, l'Hill-Climbing è piuttosto rapido, poiché è abbastanza facile migliorare uno stato sfavorevole.

Algoritmi di Ricerca Locale

L'algoritmo Hill-Climbing, alcune considerazioni e svantaggi

L'algoritmo viene talvolta chiamato *ricerca locale greedy*, poiché segue una strategia simile agli algoritmi di ricerca best-first greedy che abbiamo visto in precedenza - sceglie uno stato "buono" senza pensare a come andrà avanti.

Sfortunatamente, l'algoritmo Hill-Climbing spesso non riesce ad identificare la soluzione migliore a livello globale per una serie di ragioni.

Quindi, massimi locali, plateau e creste possono deteriorare le prestazioni degli algoritmi Hill-Climbing. In ognuno di questi casi, infatti, questi algoritmi raggiungeranno un punto dal quale non riusciranno a fare ulteriori progressi.

Statisticamente, il problema delle 8 regine viene risolto solo nel 14% dei casi. In media, tuttavia, l'algoritmo richiederà solo 4 passi per trovare una soluzione.

È da notare il fatto che lo spazio degli stati del problema delle 8 regine è pari a 8^8 , circa 17 milioni di stati. Quindi, tutto sommato, gli algoritmi Hill-Climbing non sono sempre così male.

Vale la pena cercare soluzioni per superare alcuni dei limiti che abbiamo visto.

aximum

tate space

L'algoritmo Hill-Climbing, miglioramenti

Potrebbe valere la pena di continuare la ricerca con una mossa laterale nella speranza che il plateau sia effettivamente uno spalla?

Generalmente sì, ma non possiamo consentire all'algoritmo di eseguire un numero illimitato di mosse laterali, poiché potrebbe andare in ciclo infinito una volta raggiunto un massimo locale piatto che non è una spalla.

Una soluzione diffusa è quella di stabilire un limite massimo al numero di mosse laterali. Questo chiaramente aumenta il tasso di successo dell'algoritmo, impattando però la sua efficienza.

Ad esempio, impostando un limite di 100 mosse laterali consecutive nel problema delle 8 regine, il tasso di successo arriva al 94% (dal 14% dell'algoritmo di base) ma il numero medio di mosse salirà a 21 (da 4).

Esistono diverse altre varianti dell'algoritmo di base dell'Hill-Climbing, le quali vanno sostanzialmente a lavorare sulla probabilità che l'algoritmo riesca ad uscire da un massimo locale ed esplorare lo spazio di ricerca in maniera più efficace.

Anche in questi casi, però, l'efficienza degrada. La scelta dell'algoritmo da utilizzare dipende dal tipo di problema da affrontare.

Algoritmi di Ricerca Locale

L'algoritmo Hill-Climbing, varianti

L'Hill-Climbing stocastico sceglie a caso tra tutte le mosse che vanno verso l'alto; a differenza dell'algoritmo di base, non sceglie immediatamente la mossa più conveniente, aumentando le chance di navigare lo spazio di ricerca in maniera più estesa.

L'Hill-Climbing con prima scelta può generare le mosse a caso fino a trovarne una migliore dello stato corrente. Questa strategia è molto buona quando uno stato ha molti (migliaia) successori, poiché l'algoritmo riuscirà a navigare meglio lo spazio di ricerca.

L'Hill-Climbing con riavvio casuale può consentire alla ricerca di ripartire da un punto a caso dello spazio di ricerca - è da notare che generare uno stato casuale in uno spazio degli stati può essere di per sé un problema difficile.

Il riavvio casuale consente, in pratica, all'algoritmo di avviare una serie di ricerche hill-climbing. Risulta essere completo perché prima o poi dovrà generare, come stato iniziale, proprio un obiettivo.

Più in generale, il successo dell'ultimo algoritmo dipende dalla forma del panorama dello spazio degli stati. Se ci sono pochi massimi locali e plateau, allora troverà una soluzione molto velocemente. Ma purtroppo, non è sempre così.

I problemi NP-hard hanno tipicamente un numero esponenziale di massimi locali.

Algoritmi di Ricerca Locale

L'algoritmo Simulated Annealing

Un algoritmo che non scende mai a “valle” verso stati con valore inferiore sarà sicuramente incompleto, poiché potrebbe restare bloccato in un massimo/minimo locale.

Di contro, un'esplorazione completamente casuale è completa ma estremamente inefficiente - dovrà continuamente navigare lo spazio di ricerca senza una strategia.

Sembra perciò sensato combinare le due cose, ovvero proporre una ricerca hill-climbing con un'esplorazione casuale, per migliorare sia efficienza che completezza.

Questo è quello che si propone di fare il **Simulated Annealing**.



In metallurgia, l'annealing è il processo usato per indurire i metalli o il vetro riscaldandoli ad altissime temperature e raffreddandoli gradualmente, permettendo al materiale di cristallizzare in uno stato a bassa energia.

La similitudine fu introdotta per indicare che, in una fase iniziale, l'algoritmo “scuoterà” molto la ricerca, variando molto nel panorama degli stati, per poi man mano ridurre l'intensità di questo meccanismo, concentrandosi quindi su un punto specifico del panorama e cercando una soluzione ottima in quel contesto.

Algoritmi di Ricerca Locale

L'algoritmo Simulated Annealing

```
function SIMULATED-ANNEALING(problema, velocità_raffreddamento)
    returns uno stato soluzione

    nodo_corrente ← CREA-NODO(problema.STATO-INIZIALE)

    for t ← 1 to ∞
        T ← velocità_raffreddamento[t]
        if T = 0 then return nodo_corrente
        successivo ← un successore scelto a caso di nodo_corrente
        ΔE ← successivo.VALORE - nodo_corrente.VALORE

        if ΔE > 0 then nodo_corrente ← successivo
        else nodo_corrente ← successivo solo con probabilità  $e^{\Delta E/T}$ 
```

Algoritmi di Ricerca Locale

Questo pseudocodice descrive **Simulated Annealing**, un algoritmo di ricerca locale che cerca di superare i limiti dell'Hill-Climbing. L'idea prende ispirazione dalla metallurgia: il raffreddamento controllato di un metallo (annealing) permette di raggiungere configurazioni stabili e ottimali.

Ecco come funziona:

Partenza: si parte da uno stato iniziale.

Temperatura (T): decresce col tempo secondo una funzione di raffreddamento. Più alta è T, più l'algoritmo è disposto a "rischiare".

Ad ogni passo:

Si sceglie un successore casuale dello stato corrente.

Si calcola la variazione di valore $\Delta E = \text{valore}(\text{successore}) - \text{valore}(\text{corrente})$.

Se $\Delta E > 0$ (cioè migliora), ci si sposta al successore.

Se $\Delta E \leq 0$ (peggiora), ci si può spostare lo stesso, ma solo con una certa probabilità $e^{(\Delta E/T)}$: quando T è alta la probabilità è maggiore, quando T è bassa quasi nulla.

Fine: quando $T \rightarrow 0$, il sistema si "congela" e l'algoritmo restituisce lo stato attuale.

In parole semplici:

- All'inizio l'algoritmo esplora anche soluzioni peggiori (per non bloccarsi nei massimi locali).
- Col tempo diventa più "esigente", accettando solo miglioramenti.
- Alla fine si comporta come Hill-Climbing, ma con la possibilità di essere già scappato da trappole locali.

È come un esploratore che all'inizio vaga un po' a caso, accettando anche strade in discesa, ma col passare del tempo diventa sempre più selettivo finché si ferma su un buon picco.

L'algoritmo Simulated Annealing

```
function SIMULATED-ANNEALING(problema, velocità_raffreddamento)
    returns uno stato soluzione

    nodo_corrente ← CREA-NODO(problema.STATO-INIZIALE)

    for t ← 1 to ∞
        T ← velocità_raffreddamento[t]
        if T = 0 then return nodo_corrente
        successivo ← un successore scelto a caso di nodo_corrente
         $\Delta E$  ← successivo.VALORE - nodo_corrente.VALORE

        if  $\Delta E > 0$  then nodo_corrente ← successivo
        else nodo_corrente ← successivo solo con probabilità  $e^{\Delta E/T}$ 
```

Vale la pena notare che la probabilità decresce esponenzialmente in base alla cattiva qualità della soluzione (ΔE). Inoltre, decresce anche con la temperatura T - questa scenderà costantemente ad ogni passo dell'algoritmo.

In altri termini, mosse “cattive” verranno accettate più facilmente all’inizio poiché T sarà alto e diventeranno sempre meno probabili a mano a mano che T si abbassa.

Se la temperatura si abbassa abbastanza lentamente, l'algoritmo troverà un ottimo globale con una probabilità tendente ad 1.

Algoritmi di Ricerca Locale

L'algoritmo Local Beam

Memorizzare un solo nodo può sembrare una reazione estrema ai problemi legati alle limitazioni di memoria. Una variante è rappresentata dalla beam search.

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

L'ampiezza del raggio è definita in base ad una euristica (ad esempio, $h(n)$ potrebbe rappresentare l'euristica); solo i nodi nel raggio vengono conservate e considerate come candidati per la soluzione.

La Beam Search è, quindi, a tutti gli effetti una soluzione *greedy* alla risoluzione dei problemi, poiché espanderà sempre il nodo che ritiene essere più utile al raggiungimento della soluzione.

Da un punto di vista pratico, ad ogni livello dell'albero di ricerca l'algoritmo genera tutti i successori degli stati a quel livello, ordinandoli in maniera decrescente in base all'euristica scelta. Conserverà poi solo i primi k di ogni livello, che saranno poi espansi.

Chiaramente, maggiore sarà il raggio e minore sarà il numero di nodi potati. L'algoritmo restituirà la prima soluzione identificata.

Sebbene migliori l'occupazione di memoria (in questo caso, la complessità spaziale sarà pari ad $O(kd)$), l'algoritmo *non è completo né ottimale*.

Algoritmi di Ricerca Locale

L'algoritmo Local Beam

Memorizzare un solo nodo può sembrare una reazione estrema ai problemi legati alle limitazioni di memoria. Una variante è rappresentata dalla beam search.

L'algoritmo di ricerca local beam tiene traccia di k stati anziché uno. All'inizio comincia con k stati generati casualmente: ad ogni passo, sono poi generati i successori di tutti i k stati.

Se uno di questi è un obiettivo, la ricerca termina. Altrimenti, scegli i k successori migliori dalla lista e ricomincia.

In linea teorica, potrebbe sembrare simile all'Hill-Climbing con riavvio casuale. Tuttavia, nella ricerca con riavvio casuale ogni processo di ricerca è del tutto indipendente dagli altri. Nella local beam, invece, la ricerca è "informata", nel senso che informazione viene passata dall'uno all'altro thread di ricerca paralleli.

Questo aumenta le chance di navigare il panorama degli stati verso stati più "promettenti", possibilmente aumentando le chance di trovare un ottimo globale.

Da un altro punto di vista, la local beam potrebbe soffrire di una carenza di diversificazione tra i k stati, concentrando quindi la ricerca troppo rapidamente in una piccola regione dello spazio.

Algoritmi di Ricerca Locale

L'algoritmo Local Beam

Un'alternativa è rappresentata dalla ricerca **local beam stocastica**, che è ispirata ai concetti già discussi con l'Hill-Climbing stocastico.

Invece di scegliere i k successori migliori tra i candidati, si scelgono k successori a caso. Tuttavia, ai migliori di questi si assegna una probabilità maggiore di scelta.

Questo approccio ricorda il processo di *selezione naturale*, in cui i “successori” (progenie) di uno “stato” (organismo) popolano la generazione successiva in base al loro “valore adattativo” (idoneità, anche detta *fitness*).

Nella prossima lezione, andremo ad analizzare una variante della local beam stocastica ispirata alla *teoria evolutiva di Darwin*, in cui gli stati successori sono generati come combinazione di due *stati padre* - invece di modificarne uno.

Gli algoritmi genetici considerano infatti il problema di ottimizzazione in analogia con la teoria evolutiva e simulano il processo di selezione naturale dove le configurazioni sono viste come individui di una popolazione e gli individui meno adatti “muoiono” senza riprodursi.

L'idea alla base, chiaramente, sarà quella di generare di volta in volta una generazione nel complesso migliore della precedente, aumentando le chance di identificare una soluzione ottima globale.